Recovering Token Identity from Causal Compression

Jubin Pakdel
New York University
jjp9216@nyu.edu

Abstract

Compression can be used to reduce the quadratic cost of attention in transformers, but it comes with a performance tradeoff. What linguistic information is lost when text is compressed via Language Model (LM) loss, and how can it be recovered? I study the Compress and Attend Transformer (CAT) Prakash et al. [2025], in which a bidirectional encoder compresses chunks of tokens $f(c)$ into a single vector, and a causal decoder attends to them for next-token prediction. Through linguistic probes [Conneau et al., 2018], I show that LM loss based compression retains key linguistic features of tokens necessary for next-token prediction, and discards their specific identities. The rarer a token, the less likely its recovery from the compressed representation. I train a toy CAT model, evaluating three variants with an added token reconstruction loss against a vanilla baseline. All three improve three of the five SentEval probes by roughly 3–5 percentage points and modestly improve perplexity, up to -2.5% vs the 3-epoch baseline. I run a linear cross-stitch analysis to measure how much of one representation is linearly recoverable from the other. Across the three recon variants and two corpora, the recon-trained f_r predicts vanilla f_v consistently better than the reverse ($\Delta_C \in [+0.38, +0.44]$): adding reconstruction loss encodes more linearly accessible information than LM loss alone.

1 Introduction

As transformers scale, reducing the quadratic cost of attention has become a central problem in language modeling. Compute efficient architectures each come with their own tradeoffs.

Sliding window attention Beltagy et al. [2020] restricts the range over which a token can contextualize itself, which weakens long range dependencies. Sparse attention Child et al. [2019] uses fixed masks to decide which tokens can attend to which. Linear attention Katharopoulos et al. [2020] replaces softmax with a kernelized dot product, producing a recurrent state that loses long-context recall. The Compressive Transformer Rae et al. [2020] and AutoCompressor Chevalier et al. [2023] compress past tokens into smaller representations to extend context length. These methods are typically recurrent, requiring backpropagation through time and losing parallelism.

Compress & Attend Transformer (CAT) Prakash et al. [2025] is an architecture that compresses each chunk of tokens into a single vector $f(c)$, and decodes by attending over the compressed chunks instead of the raw tokens. By encoding each chunk independently and fixing the chunk size to a power of 2, CAT avoids recurrence and allows for efficient parallelization. However, its performance on long context and needle-in-a-haystack recall tasks degrades as the chunk size 2^N grows. Since these tasks are evaluated by reading the final decoder's output, this raises the question: is information lost inside the $f(c)$ vectors, or does attention fail to retrieve information that the $f(c)$ vectors contain?

I adopt six SentEval style linguistic probes from Conneau et al. [2018] and compare CAT's compressed chunk vectors $f(c)$ to five sentence encoders that also compress text into a single vector. CAT ties or beats every reference model on four of the five syntactic and semantic probes at every chunk size $C \in \{4, 8, 16, 32\}$ on both FineWeb-Edu Penedo et al. [2024] and WikiText-103 Merity et al. [2017], and trails only LLM2Vec on SOMO by about one point. On the only token recall task, CAT's macro-F1 of 0.10–0.13 trails LLM2Vec's 0.46 by $\sim 4\times$ and BGE-large's 0.16–0.20 by $\sim 1.6\times$ fig. 1.

Preprint.

Token recall is also uneven across positions within a chunk. A position conditioned WC probe on the pretrained CAT at $C = 32$ shows the first four tokens recovered $1.65\times$ better than the last four on FineWeb-Edu and $2.81\times$ better on WikiText-103 fig. 2. Token recall also worsens as token rarity increases (as it does for every reference encoder table 13), evidenced by the WC probe's weighted F1 scoring several times higher than macro F1, which is counterintuitive to human nature. During a conversation, a person likely remembers unique words like "Albanian" long after forgetting which prepositions were used.

This problem framing extends beyond CAT. Neural image compressors de Andrade and Bajić [2024] and in-context autoencoders Ge et al. [2024] both encode based on a single training objective, producing representations that are hard to repurpose. Both studies increase their representation quality by adding a small secondary term to the loss: a β-weighted reconstruction term for the image compressor, and a language modeling term for the autoencoder.

I train an 84.4M parameter CAT from scratch on 124M FineWeb-Edu tokens at $C = 32$ and add a reconstruction penalty $\mathcal{L}_{\text{recon}} = \sum_{i,n} w_n \ell_{i,n}$ to the loss function, which forces each chunk vector to also predict the tokens it was compressed from. I ablate the weights attached to the reconstruction penalty and run multiple epochs to validate the results.

My modified CAT reduces perplexity by up to 2.5% against a 3-epoch vanilla baseline, improves WC macro-F1 by 3-5$\times$, and improves three of the five SentEval probes by 3–5 percentage points with only a 0.02% parameter increase. A linear cross-stitch test, which compares two representations by predicting one from the other, shows the reconstruction trained f_r predicts vanilla f_v consistently better than the reverse across all three recipes and both corpora.

2 Related Work

Compression-based Transformers: CAT [Prakash et al., 2025] compresses each chunk of C tokens with a bi-directional transformer, and a second transformer attends over the compressed chunks to perform next token prediction. This improves self-attention from $O(N^2)$ to $O(CN)$ and next token prediction to $O(\frac{N^2}{C})$. However, CAT has two tradeoffs: training costs roughly twice the compute, and information is lost as more tokens are compressed into each chunk. Using CAT, I study what linguistic information NTP-based compression discards.

Information Foundations: The information bottleneck (IB) [Tishby et al., 1999] characterizes a representation Z of input X as a trade-off between $I(Z, Y)$, how well Z predicts the target Y, and $I(Z, X)$, how much information Z keeps about X. At the optimum, Z keeps what is needed to predict Y and discards the rest. Both terms are Shannon mutual information, which treats information as recoverable given unlimited compute. In practice, a model recovers only what its compute and weights allow, so the information IB measures does not translate directly to what a real model can use. Xu et al. [2020] addresses this with $\mathcal{V}$-information $I_{\mathcal{V}}(X \to Y)$, which measures how much X can predict Y given a fixed predictor $\mathcal{V}$. $\mathcal{V}$-information is directional, and can be increased through finetuning or preprocessing that makes the structure more accessible.

Probing: Conneau et al. [2018] start from considering that the entire meaning of a sentence cannot be compressed into a single vector. Downstream tasks cannot definitively test this, since they require complex inference that obfuscates which information the model used. Ten simple classification tasks are created, covering surface, syntactic, and semantic information, each isolating one linguistic property, and classifiers are trained on frozen sentence embeddings to probe for that information. They find that logistic regression performs close to an MLP, so most of these properties can be read out linearly. Since CAT does not guarantee a full sentence in each chunk, which some of the probes rely on, I adopt the most relevant tasks, word content (WC), subject number, object number, tense, bigram shift, and semantic odd-man-out.

Cross-stitching We can also fit a probe from one embedding to another, training ψ to predict Z_2 from Z_1 and Z_1 from Z_2. If Z_1 is more predictive of Z_2, there is structure in Z_1 that the probe can use to recover Z_2, but not the reverse. Almudévar and Ortega [2026] formalize this concept with a directional score, which with $\mathcal{V}$ fixed to linear maps is $S^{V}_{\text{rep}}(Z_1 \to Z_2) = 1 - \inf_{\psi \in V} \text{MSE}(Z_2, \psi(Z_1))/\text{Var}(Z_2)$, the fraction of the variance in Z_2 that Z_1 explains. They find stitching is frequently asymmetric, with Z_1 predicting Z_2 while Z_2 fails to reconstruct Z_1 because of information loss. Hernandez et al. [2023] also find stitching between layers of image classifiers to be asymmetric, and show that a stitch trained only to minimize MSE against the receiver's expected representation fails to reach high task accuracy. A low reconstruction error therefore does not guarantee the receiver can use the stitched representation, so the directional asymmetry must be paired

with validation that the receiver's native task still works through the stitched representation. In conjunction, $S^V_{\text{rep}}(Z_1 \to Z_2) > S^V_{\text{rep}}(Z_2 \to Z_1)$ indicates Z_1 has more predictive power than Z_2.

3 Initial Probes

3.1 Model Selection

Five sentence embeddings were selected (table 1) as a reference for interpreting CAT's performance on linguistic probes. The four BERT-style encoders span a range of backbone sizes, training corpora, and training objectives, which provide greater variance to the probe results and additional context for interpreting CAT's metrics. LLM2Vec is the only other encoder built from a next-token prediction model in the set. All five models were created with the goal of compressing text into a single vector, whereas CAT's chunk vectors are made to be attended to for performing next-token prediction. The other encoders were trained with more steps on broader data and with objectives more explicitly oriented toward sentence compression. 1024-dim PCA projections of CAT and LLM2Vec were included to match GTE-large and BGE-large in dimensionality.

Model	Dim	Backbone	Training	Features	Ref.
CAT	2048	transformer (LM-trained)	next-token prediction	the model under study	Prakash et al. [2025]
LLM2Vec	2048	Sheared-LLaMA (1.3B)	bidir. + MNTP + supervised contrastive	LLM-derived encoder	BehnamGhader et al. [2024]
SBERT	384	MiniLM-L6 (22M, distilled)	siamese on ∼1B pairs	tiny distilled backbone	Reimers and Gurevych [2019]
SimCSE	768	BERT-base (110M)	supervised contrastive on NLI	NLI-supervised	Gao et al. [2021]
GTE-large	1024	BERT + RoPE + GLU (434M)	multi-stage contrastive	web-scale retrieval	Li et al. [2023]
BGE-large	1024	BERT-large (335M)	RetroMAE + contrastive	sentence-bottleneck pre-training	Xiao et al. [2023]

Table 1: Encoders compared. The four BERT-family encoders differ on backbone size (22M / 110M / 335M / 434M) and on contrastive-data type (NLI vs. broad sentence-pair vs. web-retrieval).

3.2 Linguistic Probes

I run the six probes on the compressed chunk vectors of every model. Each chunk is encoded with a frozen encoder, and a linear classifier is fit on top of those vectors. The probe is linear rather than an MLP, as Conneau et al. [2018] shows that they perform similarly. The cross-stitch test in §5 uses the same linear probe family, which keeps the two measurements comparable.

The six original probes are:

- **Word content (WC)**: predicts which of the top-1000 most frequently occurring words is present in a chunk. Unlike the others, this is multi-label.
- **Subject number (SubjNum)**: classifies the subject of a chunk as singular or plural.
- **Object number (ObjNum)**: classifies the object of a chunk as singular or plural.
- **Tense**: classifies the chunk's verb as present or past tense.
- **Bigram Shift**: classifies original from modified chunks, where adjacent words are swapped, e.g. "I love pizza" → "I pizza love"
- **Semantic odd-man-out (SOMO)**: classifies original from modified chunks, where a noun or verb is replaced by another from the top-1000 words, e.g. "I think pizza" → "I prefer pizza"

The five binary tasks (SubjNum, ObjNum, Tense, BShift, SOMO) use `Logistic Regression` from `scikit-learn`. WC uses a torch sigmoid probe trained with BCE because of its multi-label structure. For WC, Macro-F1 is listed, since exact accuracy on a 1000-way multi-label task is incredibly unlikely. Each probe is fit on two corpora, one in and one out of distribution (table 10). I built a separate probe dataset for each chunk size $C \in \{4, 8, 16, 32\}$, because classifying an attribute of 1 out of 32 tokens is inherently more difficult than 1 out of 4, which is evident by the decreasing performance of all evaluated models as $C \to 32$.

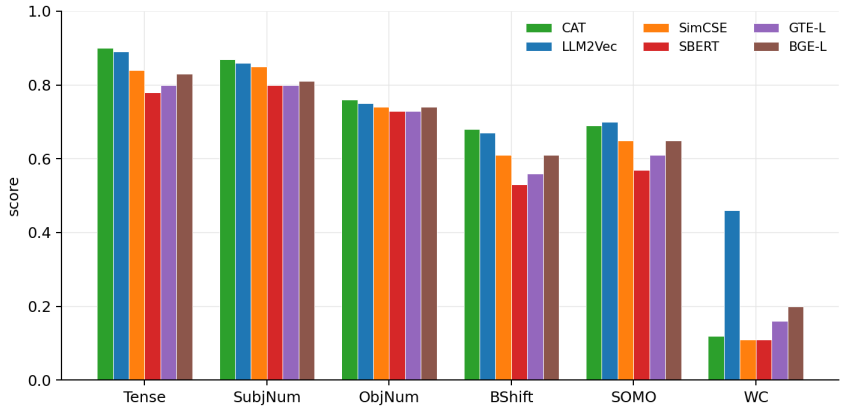


Figure 1: CAT Probe Results - loses on WC

Table 2: Linguistic probe scores across chunk sizes C, averaged across FineWeb-Edu and WikiText-103. Accuracy for the binary probes; WC reports macro-F1. Per corpus scores are in table 15 and table 16.

Task	C	CAT	LLM2VEC	simCSE	SBERT	CAT-PCA	LLM2VEC-PCA	GTE-L	BGE-L
Tense	4	0.95	0.94	0.92	0.88	0.95	0.94	0.92	0.92
	8	0.94	0.94	0.89	0.84	0.94	0.93	0.87	0.89
	16	0.93	0.92	0.86	0.80	0.93	0.90	0.82	0.84
	32	0.90	0.89	0.84	0.78	0.90	0.88	0.80	0.83
SubjNum	4	0.97	0.96	0.95	0.91	0.97	0.96	0.93	0.94
	8	0.95	0.94	0.91	0.86	0.95	0.93	0.88	0.88
	16	0.91	0.90	0.87	0.82	0.91	0.89	0.82	0.83
	32	0.87	0.86	0.85	0.80	0.87	0.85	0.80	0.81
ObjNum	4	0.92	0.90	0.88	0.83	0.92	0.90	0.86	0.87
	8	0.87	0.85	0.82	0.78	0.87	0.85	0.81	0.81
	16	0.80	0.80	0.77	0.75	0.80	0.79	0.75	0.77
	32	0.76	0.75	0.74	0.73	0.76	0.74	0.73	0.74
BShift	4	0.81	0.75	0.73	0.63	0.80	0.73	0.69	0.73
	8	0.78	0.73	0.71	0.59	0.78	0.71	0.65	0.71
	16	0.74	0.70	0.66	0.55	0.74	0.67	0.61	0.66
	32	0.68	0.67	0.61	0.53	0.68	0.63	0.56	0.61
SOMO	4	0.78	0.76	0.74	0.67	0.77	0.75	0.72	0.73
	8	0.77	0.77	0.73	0.65	0.77	0.75	0.71	0.73
	16	0.74	0.74	0.70	0.61	0.74	0.72	0.66	0.70
	32	0.69	0.70	0.65	0.57	0.69	0.66	0.61	0.65
WC	4	0.11	0.46	0.09	0.06	0.13	0.36	0.16	0.18
	8	0.10	0.46	0.07	0.06	0.10	0.34	0.13	0.16
	16	0.11	0.46	0.08	0.07	0.10	0.36	0.13	0.17
	32	0.12	0.46	0.11	0.11	0.12	0.38	0.16	0.20

3.3 Probe Results

CAT beats or matches every reference encoder on tense, subject number, object number, and bigram shift, and trails only LLM2Vec on SOMO, by about one point at the largest chunk sizes table 2. The compressed chunk vector $f(c)$ carries enough about the tokens it saw to identify their syntactic and semantic qualities. However, CAT trails LLM2Vec by $\sim 4\times$ and BGE-L by $\sim 1.6\times$ on recovering token identity (WC). Projecting CAT down to 1024 dimensions moves its score on any probe by about 0.02 at most, so its WC gap is not a product of its larger vector. Cross-entropy encodes token information that is useful for NTP and is free to discard the rest. These results suggest token identity is not consistently needed to minimize cross-entropy loss. fig. 1

The sharp split in probe performance prompted a closer look for structure in CAT's token recall. I created a new token recall probe (position-WC) with vocabulary restricted to only the bottom 200 of the top-1000 most frequently occurring words. Position-WC consists of three probes, each predicting which of those bottom

200 tokens occupy the first, middle, or last 4 positions of the chunk. On the pre-trained CAT model, accuracy on the last 4 tokens was half that on the first 4 fig. 2, table 3.

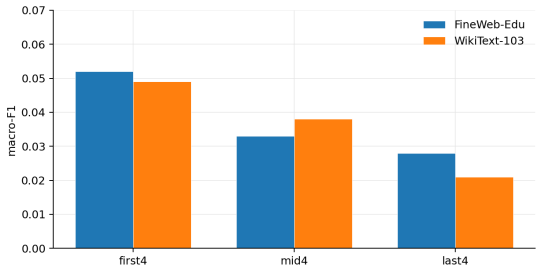


Figure 2: Position-WC on CAT-32. The beginning of a chunk is easier to predict.

Corpus	Position	acc	macro-F1	weighted-F1
fw-edu	first4	0.213	0.052	0.288
	mid4	0.108	0.033	0.168
	last4	0.129	0.028	0.158
wt-103	first4	0.206	0.049	0.281
	mid4	0.124	0.038	0.200
	last4	0.073	0.021	0.122

Table 3: Position-WC at pre-trained CAT, C=32. Each test chunk contains exactly one rare top-1000 word (ranks 801–1000) at the named within-chunk position.

I weight the position-WC F1 by token frequency and find the weighted score to be significantly higher than the macro baseline. The same result holds on the original WC probe table 13, showing that CAT is better at predicting common tokens than rare ones. Given this uneven distribution of token identity loss, I trained a toy model to test whether CAT's performance on these tasks could be improved.

4 Training a Toy CAT

4.1 Setup

The toy CAT was trained on a 124M-token subset of FineWeb-Edu with only $\sim$ 84M parameters (table 9), whereas the pre-trained CAT evaluated previously was trained adaptively over chunk sizes $\{4, 8, 16, 32\}$ on a corpus roughly two orders of magnitude larger. To study NTP-based compression at the highest magnitude, I fix the chunk size to the largest setting, 32 tokens.

I added a reconstruction objective and measured its effect on the WC probe. It is defined per position, for the n-th token within the i-th chunk, and summed across the chunk:

$$\ell_{i,n} = -\log p_\theta(t_{i,n} \mid f(c_i),\, n)\,, \qquad \mathcal{L}_{\text{recon}} = \sum_{i,n} w_n\, \ell_{i,n}$$

where w_n weights each position.

I tried two designs for p_θ:

- **Affine:** $h_n = W_n\, f(c_i) + b_n$, with a separate $W_n \in \mathbb{R}^{d\times d}$ initialized at the identity, $\sim$8.4M new parameters.
- **Bias-only:** $h_n = f(c_i) + b_n$, with only a per-position bias, $\sim$16K new parameters.

All probes were run on two corpora across 10 random seeds.

4.2 1-epoch ablations

I ablate $\lambda \in \{0.05, 0.1, 0.3, 1\}$, comparing LM loss and positional-WC between vanilla CAT and both head variants. LM perplexity falls as λ decreases for both recon variants, flattening for the affine head below λ=0.1 (fig. 3, table 11), and for $\lambda \in \{0.05, 0.1\}$, they outperform the vanilla baseline. Broadly speaking, the two heads perform similarly. At high λ the affine head does noticeably better, but the gap narrows steadily and the bias head surpasses affine at λ=0.05.

All recon variants beat the vanilla baseline on the position-WC probe (table 4). The affine variant scores between 0.82 and 1.08, the bias variant between 1.89 and 3.01. At λ=0.1 the bias head reaches an averaged macro-F1 $\sim$ 3.4$\times$ the affine head. In the affine head the weight matrix W receives some portion of the reconstruction gradient, so $f(c)$ changes less. With the lower capacity bias head, the model encodes greater token identity in $f(c)$ directly.

The naive expectation is for a larger λ to push more token identity into $f(c)$ and raise the probe score, but the results do not support this. Most of the spread across λ sizes is small enough to attribute to randomness, for

instance the affine head gains only $+0.04$ from λ=0.3 to λ=0.05, and the bias head only $+0.05$ from λ=1.0 to λ=0.3. The one clear exception is the bias head at λ=0.1, which exists outside such noise.

The pos-WC probe only measures vocabulary ranks 801 to 1000, but the reconstruction loss is averaged over every token in a chunk, so most of its gradient comes from common tokens. A probe over the 50 most common tokens might show the score rising with λ as expected.

Another interesting finding is that on FineWeb-Edu the vanilla toy CAT predicts the last tokens of a chunk better than the first, opposite of the pre-trained CAT (fig. 4). The reconstruction variants return to predicting the first tokens better, except at λ=0.05 (table 4).

It should be noted that the pos-WC probe has fixed token identities at each position. The difficulty of first4, mid4, and last4 reflects not only how useful those positions are for NTP, but also how rare the particular tokens in each position happen to be. The pre-trained and toy CAT were trained on different corpora, so the two models have encountered each token a different number of times. Therefore, token position and rarity must be confounded in these results. Because the two are intertwined, I treat the per-position weights as a general mechanism for emphasizing some subset of tokens during reconstruction, rather than a precise positional intervention. Going forward, the goal is simply to find a weighting that improves the overall probe score.

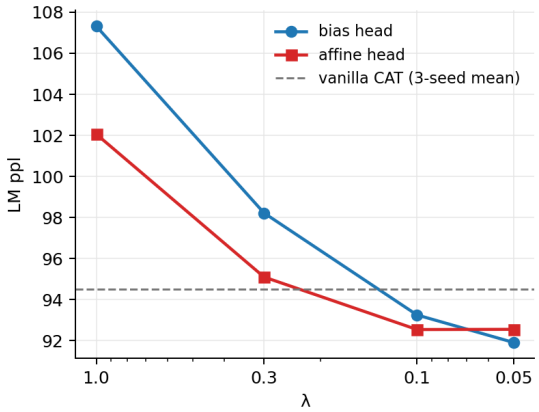


Figure 3: LM perplexity at 1 epoch. Both heads improve as λ decreases, and the affine head flattens below λ=0.1.

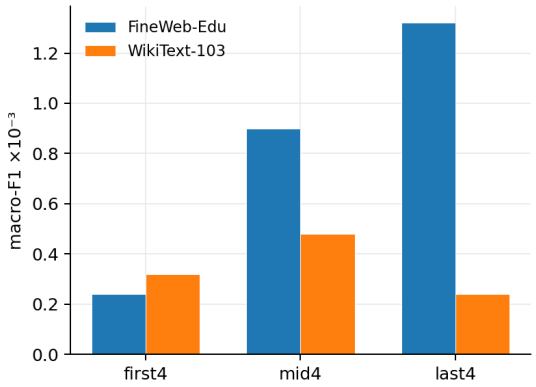


Figure 4: Vanilla toy CAT position-WC at 1 epoch. Last4 recovers better than first4 on FineWeb-Edu, opposite the pre-trained model.

Overall, the top performing ablation is the bias head at λ=0.1. It improves perplexity over the vanilla baseline, nearly matching the affine, and has the highest position-WC score. The bias head at λ=0.05 has better LM loss, but its probe score falls to roughly 60% of the λ=0.1 value, so the LM gain does not make it the better setting. The affine head could perhaps be improved with weight decay on W_n, but for the purpose of this study, I continue using the bias head. It is simpler, faster to train, and performs better.

Table 4: Position-WC macro-F1 ($\times 10^{-3}$) at 1 epoch, training seed = 42. Mean $\pm$ std over 10 probe seeds.

		FineWeb			WikiText-103			
Model	λ	first4	mid4	last4	first4	mid4	last4	avg
vanilla	—	0.24 ± 0.14	0.90 ± 0.17	1.32 ± 0.27	0.32 ± 0.24	0.48 ± 0.31	0.24 ± 0.26	0.58
affine	1.0	1.92 ± 0.16	0.63 ± 0.16	0.48 ± 0.13	2.25 ± 0.20	0.89 ± 0.15	0.29 ± 0.08	1.08
affine	0.3	1.25 ± 0.14	0.71 ± 0.15	0.90 ± 0.18	1.18 ± 0.15	0.72 ± 0.19	0.17 ± 0.09	0.82
affine	0.1	1.32 ± 0.18	0.93 ± 0.22	1.25 ± 0.20	1.14 ± 0.27	0.58 ± 0.14	0.08 ± 0.07	0.88
affine	0.05	1.05 ± 0.13	0.91 ± 0.12	1.40 ± 0.17	0.88 ± 0.28	0.71 ± 0.18	0.22 ± 0.20	0.86
bias	1.0	2.09 ± 0.34	3.22 ± 0.57	0.43 ± 0.10	2.62 ± 0.42	2.71 ± 0.76	0.28 ± 0.15	1.89
bias	0.3	2.68 ± 0.33	2.83 ± 0.36	0.63 ± 0.11	2.30 ± 0.47	2.85 ± 0.91	0.35 ± 0.21	1.94
bias	0.1	$\mathbf{2.75 \pm 0.19}$	$\mathbf{5.17 \pm 1.01}$	1.56 ± 0.18	$\mathbf{3.01 \pm 0.46}$	$\mathbf{5.00 \pm 1.24}$	0.59 ± 0.28	**3.01**
bias	0.05	1.69 ± 0.18	2.52 ± 0.56	$\mathbf{2.10 \pm 0.18}$	1.33 ± 0.41	2.71 ± 0.69	$\mathbf{1.07 \pm 0.35}$	1.90

4.3 3-epoch ablations

The best performing recon variant recovered the last four positions worst, which raises the question: how does weighting the reconstruction loss by position change CAT's probe performance?

The next ablation compares three weighting strategies:

- **last4**: $w_n = 4$ for $n \in \{28, 29, 30, 31\}$, otherwise, $w_n = 1$. Uses $\lambda = 0.1$, the best recon ablation result
- **adaptive**: (w_n is determined by an exponential moving average of the per-position losses $\ell_{i,n}$ to reduce the variance in position bias in token reconstruction, bounded by $[0.25, 4]$), and
- **uniform**: the control.

All recon variants use the cheap bias head. The adaptive weight variant uses an EMA of per-position $\ell_{i,n}$ with τ=200 steps, normalized to $w_n \in [0.25, 4]$, matching the same magnitude as the last4 weights.

Each variant ran for 3 epochs to measure how the first study's results extend with more training. Evaluation covered LM loss, position-WC, and the five linguistic probes that test non-identity features (SubjNum, ObjNum, Tense, BShift, SOMO), to see how recon affects the encoding of other linguistic features.

Both the perplexity and word-content advantages grow with more training, further supporting the previous results. last4 with λ=0.1 reduces perplexity 2.5% below vanilla table 5, and its position-WC advantage increases with the extra training, recovering rare tokens three to six times better than vanilla on FineWeb-Edu table 12. Every recon variant improves the first and middle positions, but only last4 also improves the last four, the positions CAT recovers worst fig. 5, table 6.

Table 5: LM perplexity at 3 epochs on FineWeb. Δ is relative to vanilla at 3 epochs (72.79).

weights	λ	seed	LM ppl	Δ
vanilla	—	42	72.79	—
adaptive	0.1	42	72.24	-0.8%
uniform	0.1	42	71.87	-1.3%
last4	0.1	42	70.96	-2.5%

Do these gains also carry to the original linguistic probes? If they do, reconstruction is not trading away linguistic features for token recovery. On three of the five (SubjNum, SOMO, and Tense), the leading variant beats vanilla by 3–5 percentage points, and on ObjNum by about one fig. 6, table 6. Only BShift is unchanged, and every toy model scores at chance on it.

Figure 5: Recon vs Vanilla CAT Pos-WC (3-epoch)

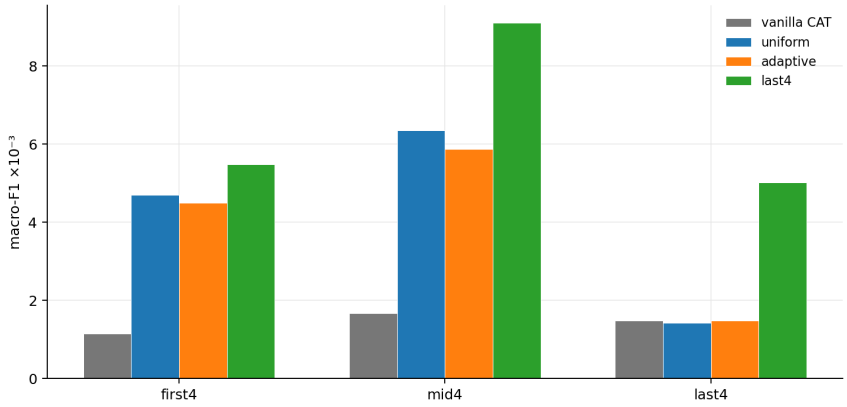
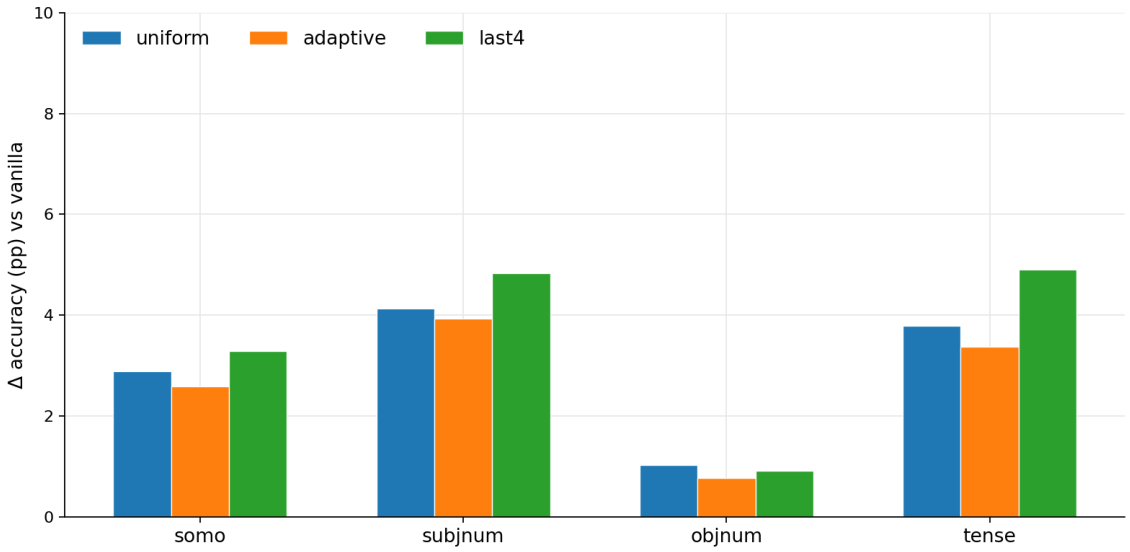


Table 6: Linear-probe performance, all three 3-epoch variants, mean over 10 probe seeds and corpus-averaged. WC reports macro-F1, and the content probes report accuracy.

	WC macro-F1 $\times 10^3$			Probes			
Recipe	first4	mid4	last4	somo	subjnum	objnum	tense
vanilla	1.15	1.67	1.48	.533	.778	.722	.779
uniform λ=0.1	4.71	6.35	1.42	.562	.819	**.732**	.817
adaptive λ=0.1	4.50	5.88	1.48	.559	.817	.730	.813
last4, λ=0.1,	**5.49**	**9.10**	**5.02**	**.566**	**.826**	.731	**.828**

last4 is the best of the three strategies on every WC cell and on three of the four content tasks, tying uniform on ObjNum table 6, while adaptive and uniform tie within seed noise. From the clean separation in probe results, it can be inferred that reconstruction benefits with a weighting strategy. A compressor improves more from a prior bias on what to keep, rather than from uniform pressure to remember everything.

Figure 6: Recon vs Vanilla CAT Linguistic Probes (3-epoch)



5 Cross-stitching

Do the recon variants f_r contain more usable information about their input tokens than the vanilla CAT f_v, or is it the same information recoded?

$\mathcal{V}$-information [Xu et al., 2020] is adapted to measure how much of one representation is linearly recoverable from the other. A linear map is fit between f_r and f_v in both directions, and the two directions are compared [Almudévar and Ortega, 2026].

For a frozen pair of embeddings, $\phi : \mathbb{R}^{512} \to \mathbb{R}^{512}$ (a 512×512 matrix and a bias) is fit on chunk pairs by MSE, trained for 5 epochs at batch size 512 with 10 probe seeds. The primary score is column-equalized R^2,

$$R^2_C(\phi) \;=\; \tfrac{1}{D}\sum_{j=1}^{D}\big(1 - \mathrm{MSE}_j/\mathrm{Var}_j\big).$$

Eq. 14 of Almudévar and Ortega [2026] writes the denominator as $\mathrm{Var}(Y)$. For this multi-variate use case, I interpret it as per-column R^2 averaged across columns (R^2_C): each output dimension is scored against its own variance, and those scores are then averaged. A map that predicts each column's mean scores 0, so R^2_C is read relative to that mean only baseline. A positive score means the probe recovers real structure, and a negative score means it does worse than the mean. Every column also counts equally, so a few high variance dimensions cannot dominate the score. The direction of Δ_C does not depend on this normalization. Δ_C was

also computed by normalizing the per-dimension scores against the total variance and against the average per-dimension variance, rather than against each dimension's own variance as in R^2_C. Neither alternative flips a sign in any cell table 14, and only the magnitude changes.

The vanilla CAT f_v is tested against the three f_r variants. Predicting f_v from f_r is consistently easier than the reverse across all three recipes and both corpora, $\Delta_C \in [+0.38, +0.44]$ table 7. Because $\mathcal{V}$ here is the family of linear maps, this asymmetry means f_r holds more linearly accessible information than f_v. f_r retains the structure f_v carries and adds to it rather than merely recoding it.

Table 7: Cross-stitch representational similarity, R^2_C, three variants. Mean over 10 probe seeds (per-cell $\sigma \approx 2{\times}10^{-4}$). $\Delta_C = R^2_C(r{\to}v) - R^2_C(v{\to}r)$.

	FineWeb-Edu			WikiText-103		
f_r recipe	$r{\to}v$	$v{\to}r$	Δ_C	$r{\to}v$	$v{\to}r$	Δ_C
Uniform, λ=0.1	.738	.335	+.403	.722	.339	+.382
Adaptive, λ=0.1	.736	.334	+.402	.721	.336	+.384
Weighted, λ=0.1, w_{last4}=4	.767	.330	+.437	.750	.336	+.414
Random-init source baselines (probe-overcapacity control):						
source $\to v$	—	−.015	—	—	+.007	—
source $\to r$ (range)	[.001, .017]	—	—	[.022, .036]	—	—

The probe is itself a trained $512{\times}512$ map, so a high R^2_C could come from its own capacity to fit a mapping rather than from structure shared between f_r and f_v. To rule this out, the same probe is fit with the source replaced by a randomly initialized, untrained network, whose outputs carry no learned structure. These score $R^2_C \approx 0$ in every cell, so the probe cannot produce a high score from a structureless input, and the trained values reflect shared structure. Hernandez et al. [2023] warns that a map trained on MSE can reach a low loss while placing its output in a region of representation space the receiver's task head cannot use. To test for this, the stitched chunk representations are passed through the receiver's own LM decoder and the perplexity ratio ratio $= \text{ppl}_{\text{native}}/\text{ppl}_{\text{stitched}} \in (0, 1]$ is reported, where a value near 0 would mean the receiver cannot read the stitched output. The ratios fall within $[0.61, 0.72]$ across all tests table 8, well above 0, so the stitched chunks remain usable to the receiver. The high cross-stitch scores therefore reflect structure, not a low MSE fit into an unreadable region.

Table 8: Cross-stitch LM-perplexity ratios, three variants. ratio $= \text{ppl}_{\text{native}}/\text{ppl}_{\text{stitched}} \in (0, 1]$, where a value near 0 would mean the receiver cannot read the stitched output.

f_r recipe	$r{\to}v$	$v{\to}r$
Uniform, λ=0.1	.695	.722
Adaptive, λ=0.1	.611	.689
Weighted, λ=0.1, w_{last4}=4	.713	.706

6 Conclusion

Taken together, the ablation, probe, and cross-stitch results prove that adding reconstruction to the language modeling loss improves compression by introducing new information useful to both the primary objective and downstream tasks, producing a richer representation.

It is worth considering the constraints and limitations of this study. All model training and probing ran on a single desktop, which kept the model, training set, and probes modest in scale. This should affect the precision, but not the general direction of the results, as they were reproduced across many probe seeds and CAT ablations.

Another consideration is how meaningful word recall, the objective of both the WC probe and reconstruction loss, really is. Does it make sense to force a transformer to memorize every input token? Probably not. It might even be beneficial to selectively forget tokens that are irrelevant to the broader meaning of a text. I posited earlier that rarer tokens can carry more meaning, and that forgetting them first is counterintuitive. This is not universally true, as common words can completely flip the meaning of a sentence. Whether something did or did not happen turns entirely on "not".

Memorizing every word equally is not the same as understanding. Reading comprehension is taught by analyzing and interpreting a text, where the task is to answer questions about it. "Who did what" and "where did it happen" have answers that map to specific tokens, the kind of recall a reconstruction objective already trains. However, some questions have no clean one-to-one mapping from tokens to an answer, like those about implied meaning or symbolism. A reader must learn which words and phrases to pay attention to and which to forget.

This learning paradigm could be translated to compression. Suppose a pre-trained language model is provided with both a text and an accompanying analysis of it. The model could then be prompted to generate an explanation, choosing which specific tokens in the original text support the analysis. That explanation serves as a preprocessing step, giving the compressor an answer key for which tokens to remember. This would force it to keep not just the common tokens, or those useful for next-token prediction, but the contextual ones that deeper comprehension depends on. The same principle carries to other learning applications. A visual classifier can already explain itself with a heatmap over the regions that drove its prediction Zhou et al. [2016] fig. 7, and those regions could act as the same answer key for what to keep. The open question then, is how do we choose what to remember? Within any input, some subset of it carries more downstream meaning, and a compressor that reconstructs it produces a more useful representation.



Figure 7: **An answer key for which parts of an input to reconstruct.** Class activation maps localize a visual question answering prediction ("skateboarding") and a mushroom classification to the most relevant regions Zhou et al. [2016].

7 Appendix

7.1 Toy CAT training

Table 9: Toy CAT architecture and training settings, shared by every run in §4 unless noted.

Decoder	4 layers, width 512, 8 heads, QK-norm
Compressor	2 layers, width 256, 4 heads
Chunk vector $f(c)$	512 dimensions
Chunk size C	32
Context length	1024 tokens
Tokenizer	GPT-2, vocabulary 50,257
Parameters	84.4M, plus 16K (bias head) or 8.4M (affine head)
Training data	124M FineWeb-Edu tokens per epoch
Batch	8 sequences of 1024 tokens
Steps	15,190 per epoch (1 epoch), 45,570 (3 epochs)
Optimizer	AdamW, $\beta = (0.9, 0.95)$, weight decay 0.1
Learning rate	6×10^{-4}, cosine decay to 6×10^{-5}
Warmup	500 steps (1 epoch), 1,500 steps (3 epochs)
Gradient clipping	1.0
Precision	bf16
Training seeds	42; also 43–44 (vanilla) and 43–45 (bias, λ=0.1) at 1 epoch

7.2 Probe settings

Table 10: Probe settings and dataset sizes. Train and test counts are per corpus and per chunk size.

Probe	Classifier	Train	Test
Pre-trained CAT and reference encoders (§3)			
Tense, SubjNum, ObjNum	`LogisticRegression`, standardized inputs, `max_iter` =1000	90,000	10,000
BShift, SOMO	`LogisticRegression`, standardized inputs, `max_iter` =1000	180,000	20,000
WC	linear layer to 1000 outputs, BCE, AdamW at 10^{-3}, 5 epochs, threshold 0.5	90,000	10,000
Position-WC	as WC, 200-word vocabulary, C=32 only	65–75k	7.3–8.3k
Toy CAT (§4), 10 probe seeds each			
Position-WC	linear layer to 1000 outputs, BCE, AdamW at 10^{-3}, 5 epochs, batch 512	65–75k	7.3–8.3k
Content probes	linear layer to 1 output, BCE, AdamW at 10^{-3}, 5 epochs, batch 512	as above	as above

7.3 1-epoch perplexity

Table 11: LM perplexity at 1 epoch on FineWeb. Δ is relative to vanilla 3-seed mean (94.50).

Model	params	λ	seed	val LM ppl	Δ
vanilla	—	—	42	94.96	+0.5%
vanilla	—	—	43	94.59	+0.1%
vanilla	—	—	44	93.94	−0.6%
vanilla (3-seed mean)	0	—	42–44	$\mathbf{94.50 \pm 0.52}$	—
affine	8.4M	1.0	42	102.06	+8.0%
affine	8.4M	0.3	42	95.10	+0.6%
affine	8.4M	0.1	42	92.54	−2.1%
affine	8.4M	0.05	42	92.55	−2.1%
bias	16K	1.0	42	107.32	+13.6%
bias	16K	0.3	42	98.22	+3.9%
bias	16K	0.1	42	93.25	−1.3%
bias	16K	0.1	43	92.81	−1.8%
bias	16K	0.1	44	92.71	−1.9%
bias	16K	0.1	45	92.72	−1.9%
bias (4-seed mean)	16K	0.1	42–45	$\mathbf{92.87 \pm 0.26}$	−1.7%
bias	16K	0.05	42	91.91	−2.7%

7.4 3-epoch position-WC by corpus

Table 12: Position-WC macro-F1 ($\times 10^{-3}$) at 3 epochs, by corpus. Mean $\pm$ std over 10 probe seeds. The adaptive recipe's per-corpus scores were not retained, so only its corpus average appears in table 6.

	FineWeb-Edu			WikiText-103		
Recipe	first4	mid4	last4	first4	mid4	last4
vanilla	1.26 ± 0.49	1.51 ± 0.36	2.33 ± 0.24	1.04 ± 0.24	1.84 ± 0.49	0.63 ± 0.21
uniform, λ=0.1	4.75 ± 0.45	6.20 ± 0.49	2.09 ± 0.22	4.67 ± 0.49	6.50 ± 1.07	0.75 ± 0.23
last4, λ=0.1	5.80 ± 0.55	9.21 ± 0.97	6.70 ± 0.83	5.18 ± 0.68	8.99 ± 0.82	3.34 ± 0.56

7.5 Word content by token frequency

Table 13: Word content weighted-F1, averaged across FineWeb-Edu and WikiText-103. Weighted-F1 weights each word by how often it occurs in the test set. Every encoder scores higher here than on macro-F1 (table 2), most by several times, so all of them recover common words better than rare ones.

C	CAT	LLM2Vec	SimCSE	SBERT	CAT-PCA	LLM2Vec-PCA	GTE-L	BGE-L
4	0.51	0.66	0.44	0.44	0.56	0.64	0.51	0.52
8	0.52	0.66	0.38	0.36	0.54	0.62	0.43	0.45
16	0.51	0.65	0.35	0.32	0.52	0.60	0.37	0.42
32	0.49	0.63	0.37	0.35	0.49	0.58	0.39	0.43

7.6 Cross-stitch normalizations

Table 14: $\Delta = R^2(r \to v) - R^2(v \to r)$ with the per-dimension error normalized three ways. The sign of Δ is positive in every cell under all three normalizations.

	FineWeb-Edu			WikiText-103		
f_r recipe	total var.	avg. per-dim var.	R^2_C	total var.	avg. per-dim var.	R^2_C
Uniform, λ=0.1	+.315	+.318	+.403	+.312	+.316	+.382
Adaptive, λ=0.1	+.324	+.307	+.402	+.323	+.308	+.384
Weighted, λ=0.1, w_{last4}=4	+.316	+.349	+.437	+.312	+.341	+.414

7.7 Probe scores by corpus

Table 15: Linguistic probe scores on FineWeb-Edu alone. Accuracy for the binary probes; WC reports macro-F1. table 2 averages this table with its counterpart.

Task	C	CAT	LLM2VEC	simCSE	SBERT	CAT-PCA	LLM2VEC-PCA	GTE-L	BGE-L
Tense	4	0.95	0.94	0.93	0.88	0.95	0.94	0.92	0.92
	8	0.94	0.94	0.89	0.83	0.94	0.93	0.87	0.88
	16	0.93	0.91	0.85	0.78	0.93	0.90	0.81	0.83
	32	0.88	0.87	0.81	0.75	0.88	0.86	0.77	0.81
SubjNum	4	0.96	0.96	0.95	0.90	0.97	0.96	0.92	0.93
	8	0.94	0.93	0.90	0.84	0.94	0.92	0.86	0.86
	16	0.90	0.89	0.85	0.79	0.90	0.88	0.79	0.81
	32	0.85	0.84	0.82	0.75	0.85	0.82	0.76	0.77
ObjNum	4	0.92	0.90	0.88	0.82	0.92	0.90	0.85	0.86
	8	0.86	0.84	0.81	0.76	0.86	0.83	0.80	0.80
	16	0.79	0.78	0.75	0.72	0.78	0.77	0.73	0.75
	32	0.73	0.73	0.71	0.70	0.73	0.72	0.71	0.71
BShift	4	0.82	0.76	0.74	0.63	0.81	0.73	0.69	0.73
	8	0.79	0.74	0.71	0.59	0.79	0.71	0.65	0.71
	16	0.75	0.71	0.67	0.55	0.75	0.67	0.61	0.66
	32	0.68	0.67	0.61	0.52	0.68	0.64	0.56	0.61
SOMO	4	0.78	0.76	0.74	0.67	0.77	0.75	0.72	0.73
	8	0.78	0.77	0.73	0.64	0.77	0.75	0.71	0.73
	16	0.74	0.74	0.70	0.61	0.74	0.72	0.66	0.70
	32	0.69	0.70	0.65	0.57	0.69	0.66	0.61	0.65
WC	4	0.11	0.44	0.08	0.06	0.13	0.36	0.15	0.18
	8	0.10	0.46	0.07	0.06	0.10	0.34	0.13	0.16
	16	0.10	0.45	0.07	0.07	0.09	0.35	0.13	0.17
	32	0.11	0.45	0.10	0.10	0.11	0.37	0.16	0.20

Table 16: Linguistic probe scores on WikiText-103 alone. Accuracy for the binary probes; WC reports macro-F1. table 2 averages this table with its counterpart.

Task	C	CAT	LLM2VEC	simCSE	SBERT	CAT-PCA	LLM2VEC-PCA	GTE-L	BGE-L
Tense	4	0.94	0.93	0.91	0.88	0.94	0.93	0.91	0.92
	8	0.94	0.93	0.89	0.85	0.94	0.93	0.87	0.90
	16	0.93	0.92	0.86	0.81	0.93	0.90	0.83	0.85
	32	0.92	0.90	0.86	0.81	0.92	0.89	0.82	0.84
SubjNum	4	0.97	0.96	0.95	0.92	0.97	0.96	0.93	0.94
	8	0.95	0.94	0.92	0.88	0.95	0.93	0.89	0.89
	16	0.92	0.91	0.89	0.85	0.92	0.90	0.85	0.85
	32	0.89	0.87	0.87	0.84	0.89	0.87	0.84	0.84
ObjNum	4	0.92	0.90	0.88	0.83	0.91	0.90	0.86	0.87
	8	0.88	0.86	0.82	0.80	0.88	0.86	0.82	0.82
	16	0.81	0.81	0.78	0.77	0.81	0.80	0.77	0.78
	32	0.78	0.77	0.76	0.75	0.78	0.76	0.75	0.76
BShift	4	0.79	0.74	0.72	0.62	0.79	0.72	0.68	0.72
	8	0.77	0.72	0.70	0.58	0.77	0.70	0.65	0.70
	16	0.73	0.69	0.65	0.55	0.73	0.66	0.60	0.66
	32	0.67	0.66	0.61	0.53	0.67	0.62	0.56	0.61
SOMO	4	0.77	0.76	0.73	0.67	0.77	0.75	0.71	0.73
	8	0.76	0.76	0.73	0.65	0.76	0.75	0.70	0.73
	16	0.73	0.74	0.70	0.60	0.73	0.71	0.66	0.69
	32	0.68	0.69	0.65	0.57	0.68	0.65	0.61	0.65
WC	4	0.11	0.47	0.09	0.06	0.13	0.35	0.16	0.18
	8	0.10	0.46	0.07	0.06	0.10	0.34	0.13	0.16
	16	0.11	0.46	0.08	0.07	0.10	0.37	0.12	0.17
	32	0.13	0.46	0.11	0.11	0.12	0.39	0.16	0.20

References

[1] A. Almudévar and A. Ortega. Bridging functional and representational similarity via usable information, Jan. 2026. URL https://arxiv.org/abs/2601.21568. Preprint, 30 January 2026.

[2] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy. LLM2Vec: Large language models are secretly powerful text encoders. In *Conference on Language Modeling (COLM)*, 2024. URL https://arxiv.org/abs/2404.05961.

[3] I. Beltagy, M. E. Peters, and A. Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. URL https://arxiv.org/abs/2004.05150.

[4] A. Chevalier, A. Wettig, A. Ajith, and D. Chen. Adapting language models to compress contexts. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://aclanthology.org/2023.emnlp-main.232/.

[5] R. Child, S. Gray, A. Radford, and I. Sutskever. Generating long sequences with sparse transformers. *arXiv preprint arXiv:1904.10509*, 2019. URL https://arxiv.org/abs/1904.10509.

[6] A. Conneau, G. Kruszewski, G. Lample, L. Barrault, and M. Baroni. What you can cram into a single $&!#* vector: Probing sentence embeddings for linguistic properties. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 2126–2136, Melbourne, Australia, 2018. Association for Computational Linguistics. doi: 10.18653/v1/P18-1198. URL https://aclanthology.org/P18-1198.

[7] A. de Andrade and I. V. Bajić. Toward task-compatible compressible representations. In *IEEE International Conference on Multimedia and Expo Workshops (ICMEW)*, 2024. URL https://arxiv.org/abs/2405.10244.

[8] T. Gao, X. Yao, and D. Chen. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2021. URL https://arxiv.org/abs/2104.08821.

[9] T. Ge, J. Hu, L. Wang, X. Wang, S.-Q. Chen, and F. Wei. In-context autoencoder for context compression in a large language model. In *International Conference on Learning Representations (ICLR)*, 2024. URL https://arxiv.org/abs/2307.06945.

[10] A. Hernandez, R. Dangovski, P. Y. Lu, and M. Soljačić. Model stitching: Looking for functional similarity between representations. *arXiv preprint arXiv:2303.11277*, 2023.

[11] A. Katharopoulos, A. Vyas, N. Pappas, and F. Fleuret. Transformers are RNNs: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning (ICML)*, 2020. URL https://arxiv.org/abs/2006.16236.

[12] Z. Li, X. Zhang, Y. Zhang, D. Long, P. Xie, and M. Zhang. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2308.03281*, 2023. URL https://arxiv.org/abs/2308.03281.

[13] S. Merity, C. Xiong, J. Bradbury, and R. Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1609.07843.

[14] G. Penedo, H. Kydíček, A. Lozhkov, M. Mitchell, C. Raffel, L. von Werra, and T. Wolf. The FineWeb datasets: Decanting the web for the finest text data at scale. In *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2024. URL https://arxiv.org/abs/2406.17557.

[15] J. Prakash, A. Puli, and R. Ranganath. Attention and compression is all you need for controllably efficient language models. *arXiv preprint arXiv:2511.05313*, 2025. URL https://arxiv.org/abs/2511.05313.

[16] J. W. Rae, A. Potapenko, S. M. Jayakumar, C. Hillier, and T. P. Lillicrap. Compressive transformers for long-range sequence modelling. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://openreview.net/forum?id=SylKikSYDH.

[17] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019. URL https://arxiv.org/abs/1908.10084.

[18] N. Tishby, F. C. Pereira, and W. Bialek. The information bottleneck method. In *Proceedings of the 37th Annual Allerton Conference on Communication, Control, and Computing*, pages 368–377, 1999. URL https://arxiv.org/abs/physics/0004057.

[19] S. Xiao, Z. Liu, P. Zhang, and N. Muennighoff. C-Pack: Packed resources for general Chinese embeddings. *arXiv preprint arXiv:2309.07597*, 2023. URL https://arxiv.org/abs/2309.07597.

[20] Y. Xu, S. Zhao, J. Song, R. Stewart, and S. Ermon. A theory of usable information under computational constraints. In *International Conference on Learning Representations (ICLR)*, 2020. URL https://openreview.net/forum?id=r1eBeyHFDH.

[21] B. Zhou, A. Khosla, A. Lapedriza, A. Oliva, and A. Torralba. Learning deep features for discriminative localization. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016.